SPACE

Endpoint Internals

Complete API Implementation Reference

Every Endpoint -- Business Logic -- Database Queries -- Security -- Side Effects

Generated from source code

SPACE Endpoint Internals

A complete reference documenting the internal implementation of every API endpoint in the SPACE media server. For each endpoint, this document describes the business logic, database queries, side effects, error conditions, and security checks that occur inside the handler.

---

1. Authentication (`/api/auth/*`)

POST /api/auth/register

Source: `src/auth/handlers.rs` -> `register()`

Flow:

1. Deserializes JSON body: `username`, `email`, `password`
2. Validates username length (>= 3) and email format (contains `@`)
3. Checks password length >= 8 characters
4. Checks `username` uniqueness via `SELECT id FROM users WHERE LOWER(username) = LOWER($1)`
5. Checks `email` uniqueness via `SELECT id FROM users WHERE LOWER(email) = LOWER($1)`
6. Hashes password with Argon2id (via `crate::utils::hash_password`)
7. Inserts new user: `INSERT INTO users (id, username, email, password_hash) VALUES ($1, $2, $3, $4)`
8. Issues a 7-day trial card via `crate::cards::issue_trial()` (partial unique index prevents duplicates)
9. Generates JWT access + refresh token pair via `crate::auth::tokens::issue_token_pair()`
10. Returns `{ access_token, refresh_token, user: { id, username, email } }`

Side Effects: Creates trial access card. No audit log entry on registration.

Error Cases: 400 if username/email already taken, password too short, invalid email format.

POST /api/auth/login

Source: `src/auth/handlers.rs` -> `login()`

Flow:

1. Deserializes JSON: `username` (string), `password` (string)
2. Looks up user: `SELECT id, password_hash FROM users WHERE LOWER(username) = LOWER($1)`
3. Verifies password with Argon2id `verify_password` (constant-time comparison)
4. On success: issues JWT pair via `issue_token_pair(user_id)`
5. Returns `{ access_token, refresh_token }`

Error Cases: 401 if user not found or password invalid. No user enumeration (same error for both).

POST /api/auth/login/totp

Source: `src/auth/handlers.rs` -> `login_totp()`

Flow:

1. Deserializes JSON: `username`, `password`, `totp_code`
2. Looks up user by username
3. Verifies password hash
4. Checks TOTP: `SELECT totp_secret FROM users WHERE id = $1`
5. Validates the 6-digit TOTP code against the secret using `totp_rs` crate (30-second window, +/-1 step tolerance)
6. If valid: issues JWT pair and returns tokens
7. If invalid: returns 401

Security: TOTP secret stored encrypted at rest. Rate-limited.

POST /api/auth/refresh

Source: `src/auth/handlers.rs` -> `refresh_token()`

Flow:

1. Deserializes JSON: `refresh_token` (string)
2. Decodes the JWT refresh token (HS256, checks expiry)
3. Extracts `sub` (user_id) and `jti` (token ID) claims
4. Checks the token ID is not in `revoked_tokens`: `SELECT 1 FROM revoked_tokens WHERE token_id = $1`
5. If not revoked: issues a new token pair (old refresh token is NOT revoked -- rotation happens on use)
6. Returns new `{ access_token, refresh_token }`

Security: Refresh tokens are stateless but revocable. Revoked tokens are stored in `revoked_tokens` table until their natural expiry.

POST /api/auth/logout

Source: `src/auth/handlers.rs` -> `logout()`

Flow:

1. Requires `CurrentUser` (extracted from Bearer token)
2. Deserializes JSON: `refresh_token` (string)
3. Decodes the refresh token to extract `jti`
4. Inserts into `revoked_tokens`: `INSERT INTO revoked_tokens (token_id, expires_at) VALUES ($1, $2)`
5. Returns `{ "logged_out": true }`

Side Effects: The refresh token is now permanently revoked.

POST /api/auth/logout-all

Source: `src/auth/handlers.rs` -> `logout_all()`

Flow:

1. Requires `CurrentUser`

2. Revokes ALL refresh tokens for the user: `DELETE FROM revoked_tokens WHERE user_id = $1` (or inserts all active token IDs)
3. In practice: sets a `token_revoked_before` timestamp on the user row, which invalidates all tokens issued before that time
4. Returns `{ "sessions_invalidated": true }`

GET /api/auth/sessions

Source: `src/auth/handlers.rs` -> `list_sessions()`

Flow:

1. Requires `CurrentUser`
2. Queries `user_sessions` table: `SELECT id, device_info, ip_address, created_at, last_active FROM user_sessions WHERE user_id = $1 ORDER BY last_active DESC`
3. Returns array of active sessions with metadata

DELETE /api/auth/sessions/{id}

Source: `src/auth/handlers.rs` -> `revoke_session()`

Flow:

1. Requires `CurrentUser`
2. Validates the session belongs to the current user
3. Deletes the session row and revokes associated refresh tokens
4. Returns `{ "revoked": true }`

POST /api/auth/password/change

Source: `src/auth/handlers.rs` -> `change_password()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `current_password`, `new_password`
3. Verifies current password against stored hash
4. Validates new password length >= 8
5. Hashes new password with Argon2id
6. Updates: `UPDATE users SET password_hash = $2, updated_at = now() WHERE id = $1`
7. Calls `logout_all()` to invalidate all existing sessions
8. Returns `{ "changed": true }`

POST /api/auth/password/forgot

Source: `src/auth/handlers.rs` -> `forgot_password()`

Flow:

1. Deserializes JSON: `email`
2. Looks up user by email (case-insensitive)
3. If user exists: generates a password reset token, stores hash in `password_resets` table with 1-hour expiry
4. If configured: sends reset email via SMTP
5. Always returns `{ "sent": true }` regardless of whether user exists (prevents email enumeration)

POST /api/auth/password/reset

Source: `src/auth/handlers.rs` -> `reset_password()`

Flow:

1. Deserializes JSON: `token`, `new_password`
2. Hashes the token and looks up in `password_resets`: `SELECT user_id FROM password_resets WHERE token_hash = $1 AND used = false AND expires_at > now()`
3. Validates new password length
4. Updates password hash on the user row
5. Marks the reset token as used
6. Calls `logout_all()`
7. Returns `{ "reset": true }`

POST /api/auth/totp/setup

Source: `src/auth/handlers.rs` -> `setup_totp()`

Flow:

1. Requires `CurrentUser`
2. Generates a new TOTP secret (32 bytes, base32 encoded)
3. Stores the secret (encrypted) on the user row: `UPDATE users SET totp_secret = $2 WHERE id = $1`
4. Returns `{ "secret": "", "qr_uri": "otpauth://totp/SPACE:?secret=...&issuer=SPACE" }`

POST /api/auth/totp/verify

Source: `src/auth/handlers.rs` -> `verify_totp()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `code` (6-digit string)
3. Validates the code against the stored TOTP secret
4. If valid: sets `totp_enabled = true` on the user row
5. Returns `{ "enabled": true }`

POST /api/auth/totp/disable

Source: `src/auth/handlers.rs` -> `disable_totp()`

Flow:

1. Requires `CurrentUser`
2. Requires password re-authentication
3. Clears TOTP: `UPDATE users SET totp_secret = NULL, totp_enabled = false WHERE id = $1`
4. Returns `{ "disabled": true }`

GET /api/auth/permissions

Source: `src/auth/handlers.rs` -> `my_permissions()`

Flow:

1. Requires `CurrentUser`
2. Queries user's role and permission set
3. Returns `{ "role": "...", "permissions": [...] }`

GET /api/auth/admin/users

Source: `src/auth/handlers.rs` -> `admin_list_users()`

Flow:

1. Requires `StaffUser` with `AdminView` permission
2. Queries users with pagination: `SELECT id, username, email, display_name, is_active, role, created_at FROM users ORDER BY created_at DESC LIMIT $1 OFFSET $2`
3. Returns `{ "users": [...], "total": N }`

PATCH /api/auth/admin/users/{id}

Source: `src/auth/handlers.rs` -> `admin_update_user()`

Flow:

1. Requires `StaffUser` with `AdminUsers` permission
2. Deserializes JSON patch: optional `role`, `is_active`, `display_name`
3. Builds dynamic UPDATE query with only provided fields
4. Executes update
5. Audit-logs the change: `admin.user_updated`
6. Returns updated user object

DELETE /api/auth/admin/users/{id}

Source: `src/auth/handlers.rs` -> `admin_delete_user()`

Flow:

1. Requires `StaffUser` with `AdminUsers` permission
2. Soft-deletes: `UPDATE users SET is_active = false, deleted_at = now() WHERE id = $1`
3. Revokes all sessions and tokens for that user
4. Audit-logs: `admin.user_deleted`
5. Returns `{ "deleted": true }`

---

2. Catalog (`/api/catalog/*`)

GET /api/catalog/items

Source: `src/catalog/handlers.rs` -> `list_items()`

Flow:

1. Requires `CurrentUser`
2. Query params: `page`, `per_page` (default 20), `media_type` (optional filter), `sort` (recent/title/rating), `search` (optional)
3. Builds SQL query dynamically based on filters
4. Base: `SELECT id, title, media_type, year, rating, poster_url, created_at FROM media_items WHERE owner = $1`
5. Adds `media_type` filter if provided
6. Adds `ILIKE` search on title if provided
7. Adds ORDER BY based on sort parameter
8. Applies LIMIT/OFFSET for pagination
9. Fetches total count for pagination metadata
10. Returns `{ "items": [...], "total": N, "page": P, "per_page": PP }`

Card Gating: Does NOT require an active access card (browsing is always allowed).

GET /api/catalog/items/{id}

Source: `src/catalog/handlers.rs` -> `get_item()`

Flow:

1. Requires `CurrentUser`
2. Queries: `SELECT * FROM media_items WHERE id = $1 AND owner = $2`
3. If not found: returns 404
4. Enriches with additional metadata:
- Fetches associated files: `SELECT * FROM media_files WHERE media_item_id = $1`
- Fetches latest metadata snapshot: `SELECT * FROM media_metadata WHERE media_item_id = $1 ORDER BY version DESC LIMIT 1`
5. Returns complete item object

POST /api/catalog/items

Source: `src/catalog/handlers.rs` -> `create_item()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `title`, `media_type` (movie/series/music/book/...), optional `year`, `description`, `poster_url`
3. Inserts: `INSERT INTO media_items (id, title, media_type, owner, year, description, poster_url) VALUES (...)`
4. Returns created item with 201 status

PATCH /api/catalog/items/{id}

Source: `src/catalog/handlers.rs` -> `update_item()`

Flow:

1. Requires `CurrentUser` (must be owner)
2. Deserializes partial JSON patch
3. Builds dynamic UPDATE with only provided fields
4. Executes update
5. Returns updated item

DELETE /api/catalog/items/{id}

Source: `src/catalog/handlers.rs` -> `delete_item()`

Flow:

1. Requires `CurrentUser` (must be owner or admin)
2. Deletes associated files, metadata, and the item itself (cascade)
3. Returns `{ "deleted": true }`

POST /api/catalog/import

Source: `src/catalog/handlers.rs` -> `import_metadata()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `media_item_id`, optional `source` (tmdb/imdb/tvdb)
3. Calls external metadata API (TMDB or similar) to fetch metadata
4. Creates metadata snapshot: `INSERT INTO media_metadata (id, media_item_id, source, data, version) VALUES (...)`
5. Updates the media item's poster_url, description, etc. from the imported data
6. Returns imported metadata

GET /api/catalog/search-external

Source: `src/catalog/handlers.rs` -> `search_external()`

Flow:

1. Requires `CurrentUser`

2. Query params: `q` (search query), `source` (tmdb/imdb), `type` (movie/tv)
3. Calls external API (TMDB search endpoint)
4. Returns results as-is (no database writes)

GET /api/catalog/stats

Source: `src/catalog/handlers.rs` -> `catalog_stats()`

Flow:

1. Requires `CurrentUser`
2. Runs aggregate queries:
- `SELECT media_type, COUNT(*) FROM media_items WHERE owner = $1 GROUP BY media_type`
- `SELECT COUNT(*) FROM media_items WHERE owner = $1`
3. Returns `{ "total": N, "by_type": { "movie": X, "series": Y, ... } }`

POST /api/catalog/items/{id}/quality

Source: `src/catalog/handlers.rs` -> `update_quality()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `quality` (string like "1080p", "4k", etc.)
3. Updates the quality field on the item
4. Returns updated item

GET /api/catalog/storage

Source: `src/catalog/handlers.rs` -> `storage_usage()`

Flow:

1. Requires `CurrentUser`
2. Queries: `SELECT SUM(file_size) FROM media_files WHERE owner = $1`
3. Returns `{ "total_bytes": N, "formatted": "12.5 GB" }`

GET /api/catalog/duplicates

Source: `src/catalog/handlers.rs` -> `find_duplicates()`

Flow:

1. Requires `CurrentUser`
2. Finds potential duplicates by matching title + media_type + year
3. `SELECT title, media_type, year, ARRAY_AGG(id) as ids FROM media_items WHERE owner = $1 GROUP BY title, media_type, year HAVING COUNT(*) > 1`
4. Returns groups of duplicate items

---

3. Search (`/api/search/*`)

GET /api/search

Source: `src/search/handlers.rs` -> `unified_search()`

Flow:

1. Requires `CurrentUser`
2. Query params: `q` (search query), `type` (optional filter), `page`, `per_page`
3. Tokenizes the query into search terms
4. Runs full-text search against `media_items` using PostgreSQL `ILIKE` or `to_tsvector`/`to_tsquery` depending on index availability
5. Searches across: title, description, cast members, genres
6. Ranks results by relevance (title match > description match > other)
7. Applies pagination
8. Returns `{ "results": [...], "total": N, "query": "..." }`

GET /api/search/suggest

Source: `src/search/handlers.rs` -> `suggest()`

Flow:

1. Requires `CurrentUser`
2. Query params: `q` (partial query, min 2 chars)
3. Searches for titles starting with or containing the query
4. Deduplicates results
5. Returns top 10 suggestions: `{ "suggestions": ["Title 1", "Title 2", ...] }`

GET /api/search/parse

Source: `src/search/handlers.rs` -> `parse_search()`

Flow:

1. Requires `CurrentUser`
2. Deserializes complex search query with filters: year range, genre, rating, media_type, cast, director
3. Builds a structured filter object
4. Does NOT execute a search -- returns the parsed filter for the client to use with other endpoints

---

4. Tracking & Lists (`/api/tracking/*`)

GET /api/tracking/progress

Source: `src/tracking/handlers.rs` -> `get_progress()`

Flow:

1. Requires `CurrentUser`
2. Query params: `media_item_id` (optional -- for single item) or returns all
3. Queries: `SELECT * FROM playback_progress WHERE user_id = $1 [AND media_item_id = $2] ORDER BY updated_at DESC`
4. Returns progress data including `position_seconds`, `duration_seconds`, `completed` flag

POST /api/tracking/progress

Source: `src/tracking/handlers.rs` -> `update_progress()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `media_item_id`, `position_seconds`, `duration_seconds`, optional `completed`
3. Upserts: `INSERT INTO playback_progress (...) ON CONFLICT (user_id, media_item_id) DO UPDATE SET position_seconds = $2, duration_seconds = $3, completed = $4, updated_at = now()`
4. Returns updated progress

GET /api/tracking/status/{id}

Source: `src/tracking/handlers.rs` -> `get_status()`

Flow:

1. Requires `CurrentUser`
2. Fetches the item's status (watching/completed/dropped/planned/on_hold)
3. `SELECT status FROM user_item_status WHERE user_id = $1 AND media_item_id = $2`
4. Returns `{ "status": "watching" }` or `{ "status": null }` if none

POST /api/tracking/status/{id}

Source: `src/tracking/handlers.rs` -> `set_status()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `status` (one of: watching, completed, dropped, planned, on_hold)
3. Upserts: `INSERT INTO user_item_status (...) ON CONFLICT (user_id, media_item_id) DO UPDATE SET status = $3, updated_at = now()`
4. Returns `{ "status": "..." }`

GET /api/tracking/rating/{id}

Source: `src/tracking/handlers.rs` -> `get_rating()`

Flow:

1. Requires `CurrentUser`
2. `SELECT rating FROM user_ratings WHERE user_id = $1 AND media_item_id = $2`
3. Returns `{ "rating": 8.5 }` or `{ "rating": null }`

POST /api/tracking/rating/{id}

Source: `src/tracking/handlers.rs` -> `set_rating()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `rating` (float, 0.0-10.0)
3. Validates range
4. Upserts rating
5. Returns `{ "rating": 8.5 }`

GET /api/tracking/favorites

Source: `src/tracking/handlers.rs` -> `get_favorites()`

Flow:

1. Requires `CurrentUser`
2. `SELECT media_item_id, created_at FROM user_favorites WHERE user_id = $1 ORDER BY created_at DESC`
3. Optionally joins with `media_items` to return full item data
4. Returns list of favorited items

POST /api/tracking/favorites/{id}

Source: `src/tracking/handlers.rs` -> `toggle_favorite()`

Flow:

1. Requires `CurrentUser`
2. Checks if already favorited
3. If not: inserts into `user_favorites`
4. If yes: removes from `user_favorites`
5. Returns `{ "favorited": true/false }`

GET /api/tracking/history

Source: `src/tracking/handlers.rs` -> `get_history()`

Flow:

1. Requires `CurrentUser`
2. Query params: `page`, `per_page`, `media_type` (optional)
3. Fetches items with status=completed or items with playback progress, ordered by last activity

4. `SELECT mi.*, uis.status, pp.position_seconds FROM media_items mi JOIN user_item_status uis ON ... LEFT JOIN playback_progress pp ON ... WHERE uis.user_id = $1 ORDER BY pp.updated_at DESC`
5. Returns paginated history

POST /api/tracking/history/{id}

Source: `src/tracking/handlers.rs` -> `add_history()`

Flow:

1. Requires `CurrentUser`
2. Inserts a history entry for the given media item with timestamp `now()`
3. Returns `{ "added": true }`

DELETE /api/tracking/history/{id}

Source: `src/tracking/handlers.rs` -> `remove_history()`

Flow:

1. Requires `CurrentUser`
2. Deletes the history entry for the given media item
3. Returns `{ "removed": true }`

GET /api/tracking/lists

Source: `src/tracking/handlers.rs` -> `get_lists()`

Flow:

1. Requires `CurrentUser`
2. `SELECT * FROM user_lists WHERE user_id = $1 ORDER BY name`
3. Returns array of user-created lists (name, description, item count, created_at)

POST /api/tracking/lists

Source: `src/tracking/handlers.rs` -> `create_list()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `name`, optional `description`, `is_public`
3. Inserts into `user_lists`
4. Returns created list with 201

DELETE /api/tracking/lists/{id}

Source: `src/tracking/handlers.rs` -> `delete_list()`

Flow:

1. Requires `CurrentUser` (must own the list)
2. Deletes list and all `list_items` associations (cascade)
3. Returns `{ "deleted": true }`

POST /api/tracking/lists/{list_id}/items/{item_id}

Source: `src/tracking/handlers.rs` -> `add_to_list()`

Flow:

1. Requires `CurrentUser`
2. Verifies list ownership
3. Checks item not already in list
4. Inserts into `list_items` with `position` = next available
5. Returns `{ "added": true }`

DELETE /api/tracking/lists/{list_id}/items/{item_id}

Source: `src/tracking/handlers.rs` -> `remove_from_list()`

Flow:

1. Requires `CurrentUser`
2. Verifies list ownership
3. Removes from `list_items`
4. Returns `{ "removed": true }`

GET /api/tracking/recent

Source: `src/tracking/handlers.rs` -> `get_recent()`

Flow:

1. Requires `CurrentUser`
2. Returns recently accessed items (last 24 hours) with progress
3. `SELECT mi.*, pp.position_seconds, pp.updated_at FROM media_items mi JOIN playback_progress pp ON ... WHERE pp.user_id = $1 AND pp.updated_at > now() - interval '24 hours' ORDER BY pp.updated_at DESC LIMIT 20`
4. Returns array of recent items

---

5. Streaming (`/api/streaming/*`)

GET /api/streaming/stream/{id}

Source: `src/streaming/handlers.rs` -> `stream_media()`

Flow:

1. Requires `CurrentUser`
2. **Card gate:** calls `ensure_card_access(pool, user_id)` -- blocks playback if no active card
3. Looks up the media item and its associated file(s)
4. Resolves the file path on disk via `media_files` table
5. Detects content type from file extension (mp4, mkv, avi, etc.)
6. Handles `Range` header for partial content (HTTP 206):
- Parses `Range: bytes=0-1023` format
- Seeks to the start position
- Reads up to the requested range or chunk size
- Returns `Content-Range: bytes 0-1023/total` header
7. If no Range header: streams the full file with `Transfer-Encoding: chunked`
8. Sets appropriate `Content-Type`, `Accept-Ranges: bytes`, `Content-Length` headers
9. Records streaming activity: `INSERT INTO streaming_logs (user_id, media_item_id, ip_address, user_agent, started_at) VALUES (...)`
10. Returns the file stream

Metrics: Increments `stream_started_total` counter.

Security: `streaming_protection` module may inject silent watermarks for forensic tracking.

GET /api/streaming/stream/{id}/manifest.mpd

Source: `src/streaming/handlers.rs` -> `dash_manifest()`

Flow:

1. Requires `CurrentUser`
2. **Card gate:** `ensure_card_access()`
3. Generates a DASH (MPEG-DASH) MPD manifest on-the-fly from the media item's files
4. Lists available quality levels (based on file analysis or stored metadata)
5. Returns XML manifest with `Content-Type: application/dash+xml`

GET /api/streaming/subtitles/{id}

Source: `src/streaming/subtitles.rs` -> `get_subtitles()`

Flow:

1. Requires `CurrentUser`
2. Queries: `SELECT * FROM subtitle_tracks WHERE media_item_id = $1 ORDER BY language`
3. Returns array of available subtitle tracks with language, format (srt/vtt/ass), and file path
4. Client can then fetch individual subtitle file via `GET /api/streaming/subtitles/{id}/{track_id}`

GET /api/streaming/subtitles/{id}/{track_id}

Source: `src/streaming/subtitles.rs` -> `serve_subtitle()`

Flow:

1. Requires `CurrentUser`
2. Looks up the subtitle track and resolves file path
3. Reads and returns the subtitle file with appropriate Content-Type (text/vtt, application/x-subrip, etc.)
4. Sets `Content-Disposition: inline` for browser rendering

GET /api/streaming/lyrics/{id}

Source: `src/streaming/handlers.rs` -> `get_lyrics()`

Flow:

1. Requires `CurrentUser`
2. Looks up lyrics for the given media item (from metadata or lyrics database)
3. Returns `{ "lyrics": "...", "language": "en" }` or 404 if no lyrics available

GET /api/streaming/download-url/{id}

Source: `src/streaming/download.rs` -> `generate_download_url()`

Flow:

1. Requires `CurrentUser`
2. **Card gate:** `ensure_card_access()`
3. Generates a time-limited signed download URL (HMAC-SHA256, expires in 1 hour)
4. The URL includes: `user_id`, `media_item_id`, `expiry` timestamp, `signature`
5. Returns `{ "url": "https://...?token=...", "expires_at": "..." }`

POST /api/streaming/download/{id}

Source: `src/streaming/download_handlers.rs` -> `request_download()`

Flow:

1. Requires `CurrentUser`
2. **Card gate:** `ensure_card_access()`
3. Validates the download is allowed (not DRM-protected, download enabled for this item)
4. Generates a download token (short-lived, single use)
5. Returns the file as an attachment (`Content-Disposition: attachment`)

GET /api/streaming/offline-token

Source: `src/streaming/download.rs` -> `generate_offline_token()`

Flow:

1. Requires `CurrentUser`
2. **Card gate:** `ensure_card_access()`

3. Generates a longer-lived offline playback token (7 days) that can be used without network
4. Token is signed with HMAC and includes allowed media_item_ids
5. Returns `{ "token": "...", "expires_at": "...", "allowed_items": [...] }`

GET /api/streaming/media/{id}/quality-options

Source: `src/streaming/probe.rs` -> `get_quality_options()`

Flow:

1. Requires `CurrentUser`
2. Looks up pre-probed quality metadata for the media item (stored during library scan)
3. If not available: runs ffprobe on the file to detect available streams, resolutions, codecs
4. Caches the result for future requests
5. Returns `{ "qualities": [{ "height": 1080, "codec": "h264", "bitrate": "5000k", ... }] }`

POST /api/streaming/media/{id}/transcode

Source: `src/streaming/handlers.rs` -> `request_transcode()`

Flow:

1. Requires `CurrentUser`
2. **Card gate:** `ensure_card_access()`
3. Deserializes JSON: `target_quality` (height), `target_codec` (h264/h265)
4. Checks if a transcode already exists or is in progress
5. Creates a transcoding job: `INSERT INTO transcode_jobs (id, media_item_id, user_id, target_quality, status) VALUES (...)`
6. Notifies the background worker (via channel or database polling)
7. Returns `{ "job_id": "...", "status": "queued" }`

GET /api/streaming/transcode/{job_id}

Source: `src/streaming/handlers.rs` -> `transcode_status()`

Flow:

1. Requires `CurrentUser`
2. Queries the transcode job status from `transcode_jobs`
3. Returns `{ "status": "processing", "progress_percent": 45, "eta_seconds": 120 }`

GET /api/streaming/hls/{id}/master.m3u8

Source: `src/streaming/handlers.rs` -> `hls_master()`

Flow:

1. Requires `CurrentUser`

2. **Card gate:** `ensure_card_access()`
3. Generates HLS master playlist from available quality variants
4. Returns `#EXTM3U` with `#EXT-X-STREAM-INF` entries for each quality level
5. Sets `Content-Type: application/vnd.apple.mpegurl`

GET /api/streaming/hls/{id}/{quality}/playlist.m3u8

Source: `src/streaming/handlers.rs` -> `hls_variant()`

Flow:

1. Requires `CurrentUser`
2. Returns the segment playlist for the specified quality variant
3. Lists all `.ts` segments with durations
4. Returns `Content-Type: application/vnd.apple.mpegurl`

---

6. Social (`/api/social/*`)

GET /api/social/reviews/{item_id}

Source: `src/social/handlers.rs` -> `get_reviews()`

Flow:

1. Requires `CurrentUser`
2. Queries reviews for the given media item: `SELECT r.*, u.username, u.display_name FROM reviews r JOIN users u ON u.id = r.user_id WHERE r.media_item_id = $1 ORDER BY r.created_at DESC`
3. Calculates average rating: `SELECT AVG(rating), COUNT(*) FROM reviews WHERE media_item_id = $1`
4. Returns `{ "reviews": [...], "avg_rating": 7.8, "review_count": 12 }`

POST /api/social/reviews/{item_id}

Source: `src/social/handlers.rs` -> `create_review()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `rating` (1-10), `body` (text, max 5000 chars), optional `contains_spoilers`
3. Checks user hasn't already reviewed this item
4. Inserts: `INSERT INTO reviews (id, user_id, media_item_id, rating, body, contains_spoilers) VALUES (...)`
5. Returns created review with 201

DELETE /api/social/reviews/{item_id}

Source: `src/social/handlers.rs` -> `delete_review()`

Flow:

1. Requires `CurrentUser`
2. Verifies the user owns the review
3. Deletes: `DELETE FROM reviews WHERE user_id = $1 AND media_item_id = $2`
4. Returns `{ "deleted": true }`

POST /api/social/reactions/{review_id}

Source: `src/social/handlers.rs` -> `toggle_reaction()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `reaction_type` (like/helpful/funny)
3. Checks if user already reacted with this type
4. If not: inserts reaction, increments `reaction_count` on the review
5. If yes: removes reaction, decrements count
6. Returns `{ "reacted": true/false, "count": N }`

GET /api/social/comments/{item_id}

Source: `src/social/handlers.rs` -> `get_comments()`

Flow:

1. Requires `CurrentUser`
2. Fetches comments for a media item with nested replies (max 2 levels deep)
3. Joins with users for author info
4. Returns threaded comment tree

POST /api/social/comments/{item_id}

Source: `src/social/handlers.rs` -> `create_comment()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `body` (text, max 2000 chars), optional `parent_comment_id` (for replies)
3. Validates parent comment exists if provided
4. Inserts comment
5. Returns created comment

GET /api/social/followers

Source: `src/social/handlers.rs` -> `get_followers()`

Flow:

1. Requires `CurrentUser`

2. Queries users following the current user
3. Returns list of follower user objects

GET /api/social/following

Source: `src/social/handlers.rs` -> `get_following()`

Flow:

1. Requires `CurrentUser`
2. Queries users the current user follows
3. Returns list of following user objects

POST /api/social/follow/{user_id}

Source: `src/social/handlers.rs` -> `toggle_follow()`

Flow:

1. Requires `CurrentUser`
2. Prevents self-follow
3. Checks if already following
4. If not: inserts into `user_follows`, increments follower/following counts
5. If yes: removes follow, decrements counts
6. Creates a notification for the followed user: `INSERT INTO notifications (user_id, type, message) VALUES ($1, 'follow', '...')`
7. Returns `{ "following": true/false }`

GET /api/social/activity

Source: `src/social/handlers.rs` -> `activity_feed()`

Flow:

1. Requires `CurrentUser`
2. Fetches activity from followed users (last 7 days)
3. Activity types: reviews, status changes, favorites, follows
4. `SELECT a.*, u.username FROM activity_feed a JOIN users u ON u.id = a.user_id WHERE a.user_id IN (SELECT following_id FROM user_follows WHERE user_id = $1) AND a.created_at > now() - interval '7 days' ORDER BY a.created_at DESC LIMIT 50`
5. Returns activity feed items

POST /api/social/share/{item_id}

Source: `src/social/handlers.rs` -> `share_item()`

Flow:

1. Requires `CurrentUser`
2. Generates a shareable link (public, time-limited)
3. Returns `{ "share_url": "https://.../s/abc123" }`

---

7. Chat (`/api/chat/*`)

GET /api/chat/conversations

Source: `src/chat/handlers.rs` -> `list_conversations()`

Flow:

1. Requires `CurrentUser`
2. Fetches all conversations the user participates in
3. Includes last message preview, unread count, other participant info
4. Returns sorted by last message time

POST /api/chat/conversations

Source: `src/chat/handlers.rs` -> `create_conversation()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `participant_ids` (array of user IDs)
3. Checks for existing DM conversation between these users
4. If exists: returns existing conversation
5. If new: creates `conversations` row and `conversation_members` rows
6. Returns created conversation

GET /api/chat/conversations/{id}/messages

Source: `src/chat/handlers.rs` -> `list_messages()`

Flow:

1. Requires `CurrentUser`
2. Verifies user is a member of the conversation
3. Query params: `before` (cursor for pagination), `limit` (default 50)
4. Fetches messages with cursor-based pagination (older messages)
5. `SELECT m.*, u.username FROM messages m JOIN users u ON u.id = m.sender_id WHERE m.conversation_id = $1 AND m.created_at < $2 ORDER BY m.created_at DESC LIMIT $3`
6. Returns `{ "messages": [...], "has_more": true/false }`

POST /api/chat/conversations/{id}/messages

Source: `src/chat/handlers.rs` -> `send_message()`

Flow:

1. Requires `CurrentUser`
2. Verifies user is a member
3. Deserializes JSON: `body` (text, max 10000 chars), optional `reply_to_message_id`
4. Inserts message: `INSERT INTO messages (id, conversation_id, sender_id, body, reply_to) VALUES (...)`
5. Updates conversation's `last_message_at` and `last_message_preview`
6. Increments unread count for other members
7. Sends WebSocket notification to conversation members via hub
8. Returns created message

POST /api/chat/conversations/{id}/read

Source: `src/chat/handlers.rs` -> `mark_read()`

Flow:

1. Requires `CurrentUser`
2. Updates `last_read_at` for the user in `conversation_members`
3. Resets unread count to 0
4. Returns `{ "read": true }`

POST /api/chat/conversations/{id}/typing

Source: `src/chat/handlers.rs` -> `typing_indicator()`

Flow:

1. Requires `CurrentUser`
2. Sends a WebSocket "typing" event to other conversation members
3. No database write (ephemeral)
4. Returns 200

DELETE /api/chat/messages/{id}

Source: `src/chat/handlers.rs` -> `delete_message()`

Flow:

1. Requires `CurrentUser`
2. Verifies the user sent the message
3. Soft-deletes: `UPDATE messages SET deleted = true, body = '[deleted]' WHERE id = $1`
4. Sends WebSocket update to conversation
5. Returns `{ "deleted": true }`

---

8. Parties (`/api/parties/*`)

POST /api/parties

Source: `src/parties/handlers.rs` -> `create_party()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `media_item_id`, optional `max_members`
3. Creates party: `INSERT INTO watch_parties (id, host_id, media_item_id, code, max_members) VALUES (...)`
4. Generates a short join code (6 alphanumeric characters)
5. Adds the host as a member
6. Returns party details with join code

GET /api/parties/{id}

Source: `src/parties/handlers.rs` -> `get_party()`

Flow:

1. Requires `CurrentUser`
2. Fetches party details, member list, current playback state
3. Returns party object with members and sync status

POST /api/parties/join/{code}

Source: `src/parties/handlers.rs` -> `join_party()`

Flow:

1. Requires `CurrentUser`
2. Looks up party by join code
3. Checks not at max capacity
4. Checks user is not already in the party
5. Inserts member record
6. Notifies existing members via WebSocket
7. Returns party details

POST /api/parties/{id}/leave

Source: `src/parties/handlers.rs` -> `leave_party()`

Flow:

1. Requires `CurrentUser`
2. Removes user from `party_members`
3. If user was host and party has other members: transfers host to next member
4. If last member: deletes the party
5. Notifies remaining members
6. Returns `{ "left": true }`

POST /api/parties/{id}/sync

Source: `src/parties/handlers.rs` -> `sync_playback()`

Flow:

1. Requires `CurrentUser` (must be host)
2. Deserializes JSON: `position_seconds`, `playing` (bool), `timestamp` (host's clock)
3. Updates party's sync state
4. Broadcasts sync event to all members via WebSocket
5. Members adjust their playback position to match

POST /api/parties/{id}/chat

Source: `src/parties/handlers.rs` -> `party_chat()`

Flow:

1. Requires `CurrentUser` (must be member)
2. Deserializes JSON: `message` (max 500 chars)
3. Inserts into `party_chat_messages`
4. Broadcasts chat message to all members via WebSocket
5. Returns created message

---

9. Playback (`/api/playback/*`)

GET /api/playback/sessions

Source: `src/playback/handlers.rs` -> `list_sessions()`

Flow:

1. Requires `CurrentUser`
2. Fetches active playback sessions for the user
3. `SELECT * FROM playback_sessions WHERE user_id = $1 AND ended_at IS NULL ORDER BY last_active DESC`
4. Returns array of sessions with device info, media item, position

---

POST /api/playback/sessions

Source: `src/playback/handlers.rs` -> `start_session()`

Flow:

1. Requires `CurrentUser`
2. **Card gate:** `ensure_card_access()`
3. Deserializes JSON: `media_item_id`, optional `device_info`, `quality`
4. Closes any existing sessions for this device (one session per device)
5. Creates session: `INSERT INTO playback_sessions (id, user_id, media_item_id, device_info, quality) VALUES (...)`
6. Records in `streaming_logs`

7. Returns session object

PUT /api/playback/sessions/{id}

Source: `src/playback/handlers.rs` -> `update_session()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `position_seconds`, optional `bitrate`, `quality`
3. Updates session: `UPDATE playback_sessions SET position_seconds = $2, last_active = now() WHERE id = $1 AND user_id = $3`
4. Also upserts `playback_progress` for resume tracking
5. Returns updated session

DELETE /api/playback/sessions/{id}

Source: `src/playback/handlers.rs` -> `end_session()`

Flow:

1. Requires `CurrentUser`
2. Sets `ended_at = now()` on the session
3. Finalizes `playback_progress`
4. Returns `{ "ended": true }`

GET /api/playback/resume/{item_id}

Source: `src/playback/handlers.rs` -> `get_resume_point()`

Flow:

1. Requires `CurrentUser`
2. Queries: `SELECT position_seconds, duration_seconds FROM playback_progress WHERE user_id = $1 AND media_item_id = $2 AND completed = false`
3. If position > 90% of duration: considers it completed, returns null
4. Returns `{ "position_seconds": 1234, "duration_seconds": 5432 }` or `{ "position_seconds": null }`

GET /api/playback/markers/{item_id}

Source: `src/playback/handlers.rs` -> `get_markers()`

Flow:

1. Requires `CurrentUser`
2. Fetches intro/skip markers for the media item
3. `SELECT * FROM playback_markers WHERE media_item_id = $1 ORDER BY start_seconds`
4. Returns array of markers: `[{ "type": "intro", "start": 0, "end": 90 }, { "type": "skip", "start": 1200, "end": 1230 }]`

POST /api/playback/markers/{item_id}

Source: `src/playback/handlers.rs` -> `set_markers()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: array of markers `[{ "type": "...", "start": N, "end": M }]`
3. Deletes existing markers for the item
4. Inserts new markers
5. Returns saved markers

GET /api/playback/offline-items

Source: `src/playback/handlers.rs` -> `get_offline_items()`

Flow:

1. Requires `CurrentUser`
2. Fetches items that have been downloaded for offline use
3. Returns list of media items with download status

---

10. Platform (`/api/platform/*`)

GET /api/platform/settings

Source: `src/platform/handlers.rs` -> `get_settings()`

Flow:

1. Requires `CurrentUser`
2. Fetches user's platform settings from `user_settings` table
3. Returns settings object: `{ "theme": "dark", "language": "en", "notifications_enabled": true, ... }`

PUT /api/platform/settings

Source: `src/platform/handlers.rs` -> `update_settings()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: partial settings patch
3. Upserts: `INSERT INTO user_settings (user_id, ...) VALUES (...) ON CONFLICT (user_id) DO UPDATE SET ...`
4. Returns updated settings

GET /api/platform/devices

Source: `src/platform/handlers.rs` -> `list_devices()`

Flow:

1. Requires `CurrentUser`
2. `SELECT * FROM user_devices WHERE user_id = $1 ORDER BY last_seen DESC`
3. Returns array of registered devices with name, type, last_seen, is_active

---

POST /api/platform/devices

Source: `src/platform/handlers.rs` -> `register_device()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `name`, `type` (web/ios/android/tv/desktop)
3. Upserts device (by fingerprint or creates new)
4. Returns device record

DELETE /api/platform/devices/{id}

Source: `src/platform/handlers.rs` -> `remove_device()`

Flow:

1. Requires `CurrentUser`
2. Verifies device belongs to user
3. Deletes device
4. Revokes any associated tokens
5. Returns `{ "removed": true }`

GET /api/platform/notifications

Source: `src/platform/handlers.rs` -> `list_notifications()`

Flow:

1. Requires `CurrentUser`
2. `SELECT * FROM notifications WHERE user_id = $1 ORDER BY created_at DESC LIMIT 50`
3. Returns notifications with read/unread status

POST /api/platform/notifications/{id}/read

Source: `src/platform/handlers.rs` -> `mark_notification_read()`

Flow:

1. Requires `CurrentUser`
2. `UPDATE notifications SET read = true WHERE id = $1 AND user_id = $2`

3. Returns `{ "read": true }`

POST /api/platform/notifications/read-all

Source: `src/platform/handlers.rs` -> `mark_all_read()`

Flow:

1. Requires `CurrentUser`
2. `UPDATE notifications SET read = true WHERE user_id = $1 AND read = false`
3. Returns `{ "count": N }` (number marked)

GET /api/platform/presence

Source: `src/platform/handlers.rs` -> `get_presence()`

Flow:

1. Requires `CurrentUser`
2. Returns the user's current presence status and online status
3. Presence data is stored in-memory (via Redis or tokio broadcast) with TTL-based expiry

---

PUT /api/platform/presence

Source: `src/platform/handlers.rs` -> `update_presence()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `status` (online/away/dnd/offline), optional `custom_message`
3. Updates presence in the in-memory store
4. Broadcasts presence change to followers via WebSocket

GET /api/platform/behavior

Source: `src/platform/handlers.rs` -> `get_behavior()`

Flow:

1. Requires `CurrentUser`
2. Returns user behavior settings: auto-play next, skip intro, resume from last position, etc.
3. `SELECT * FROM user_behavior WHERE user_id = $1`

PUT /api/platform/behavior

Source: `src/platform/handlers.rs` -> `update_behavior()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: behavior settings patch
3. Upserts behavior settings
4. Returns updated settings

GET /api/platform/search-history

Source: `src/platform/handlers.rs` -> `get_search_history()`

Flow:

1. Requires `CurrentUser`
2. `SELECT * FROM search_history WHERE user_id = $1 ORDER BY created_at DESC LIMIT 20`
3. Returns recent search queries

POST /api/platform/search-history

Source: `src/platform/handlers.rs` -> `add_search_history()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `query` (search string)
3. Inserts into `search_history` (deduplicates by checking if same query exists within last hour)
4. Returns `{ "added": true }`

GET /api/platform/export

Source: `src/platform/handlers.rs` -> `export_data()`

Flow:

1. Requires `CurrentUser`
2. Gathers all user data: profile, watch history, ratings, reviews, settings, lists
3. Formats as JSON
4. Returns the data as a downloadable JSON file (`Content-Disposition: attachment`)

---

11. Recommendations (`/api/recommendations/*`)

GET /api/recommendations

Source: `src/recommendations/handlers.rs` -> `get_recommendations()`

Flow:

1. Requires `CurrentUser`

2. **Card gate:** `ensure_card_access()`
3. Checks cache first: `SELECT data FROM recommendation_cache WHERE user_id = $1 AND expires_at > now()`
4. If cache hit: returns cached recommendations
5. If cache miss: runs the recommendation algorithm:

a. Fetches user's ratings, completed items, favorites

b. Finds similar users (collaborative filtering)

c. Calculates item scores based on: user ratings (weight 0.4), popularity (0.2), recency (0.1), genre similarity (0.2), collaborative signals (0.1)

d. Excludes already-watched items

e. Sorts by score descending

6. Caches result for 6 hours: `INSERT INTO recommendation_cache (user_id, data, expires_at) VALUES (...)`
7. Returns `{ "recommendations": [{ "item": {...}, "score": 0.92, "reason": "Because you liked X" }] }`

GET /api/recommendations/similar/{item_id}

Source: `src/recommendations/handlers.rs` -> `get_similar()`

Flow:

1. Requires `CurrentUser`
2. Looks up the media item's metadata (genres, tags, cast, director)
3. Finds items with similar metadata attributes
4. Scores by overlap count weighted by metadata importance
5. Returns top 10 similar items

GET /api/recommendations/home

Source: `src/recommendations/handlers.rs` -> `home_page()`

Flow:

1. Requires `CurrentUser`
2. Generates home page "rails" (horizontal rows):

- "Continue Watching" -- items with incomplete playback progress
- "Top Picks" -- personalized recommendations
- "Recently Added" -- newest items in library

- "Trending" -- most-watched across all users
- Genre-specific rails based on user preferences

3. Each rail has a title and array of items
4. Returns `{ "rails": [{ "title": "Continue Watching", "items": [...] }, ...] }`

---

12. Files (`/api/files/*`)

POST /api/files/upload

Source: `src/files/handlers.rs` -> `upload_file()`

Flow:

1. Requires `CurrentUser`
2. Receives multipart form data
3. Validates file type, size (configurable max)
4. Generates a unique filename, computes SHA-256 hash
5. Writes file to the configured storage path
6. Inserts record: `INSERT INTO media_files (id, media_item_id, filename, path, file_size, mime_type, checksum) VALUES (...)`
7. Triggers background metadata extraction (ffprobe) via tokio task
8. Returns file metadata

GET /api/files/{id}

Source: `src/files/handlers.rs` -> `get_file()`

Flow:

1. Requires `CurrentUser`
2. Fetches file record from `media_files`
3. Returns file metadata (path, size, type, checksum)

DELETE /api/files/{id}

Source: `src/files/handlers.rs` -> `delete_file()`

Flow:

1. Requires `CurrentUser` (must be owner)
2. Deletes physical file from disk
3. Deletes record from `media_files`
4. Returns `{ "deleted": true }`

GET /api/files/{id}/info

Source: `src/files/handlers.rs` -> `file_info()`

Flow:

1. Requires `CurrentUser`
2. Runs ffprobe on the file to get detailed codec info
3. Caches the probe result
4. Returns `{ "duration": 7200, "codec": "h264", "resolution": "1920x1080", "bitrate": "5000k", ... }`

---

13. Cards & Billing (`/api/cards/*`, `/api/merchant/*`)

POST /api/cards/activate

Source: `src/cards.rs` -> `activate_card()`

Flow:

1. Requires `CurrentUser`
2. **Brute-force protection:** checks `card_activation_locks` table -- if user has >= 5 failed attempts in 15 minutes, returns 429 with retry-after
3. Validates card number format: `SPC-{12 hex}-{8 hex}` (max 32 chars, alphanumeric + dash)
4. Opens a transaction with `SELECT ... FOR UPDATE` on the card row (row lock prevents concurrent activation races)
5. Looks up card by `card_number`
6. Returns identical 404 for unknown cards AND cards not owned by the caller (prevents card number enumeration)
7. Checks card status:

- `"unused"`: activates -- sets `status='active'`, computes `expires_at = now() + duration_days`
- `"active"`: allows re-activation (refreshes expiry)
- `"held"`: returns "card is held by administration"
- other: returns "card already fully consumed"

8. Commits transaction
9. Clears activation failure counter on success
10. Logs audit event: `card.activated`
11. Increments `cards_activated_total` metric
12. Returns `{ "activated": true, "expires_at_unix": N, "days_remaining": N }`

POST /api/cards/transfer-code

Source: `src/cards.rs` -> `create_transfer_code()`

Flow:

1. Requires `CurrentUser`
2. Generates random hex code: `XFER-{20 hex}`
3. Computes SHA-256 hash of the code (stored, not plaintext)
4. Inserts: `INSERT INTO card_transfer_codes (id, code_hash, requester, expires_at) VALUES (...)`
5. Code expires in 20 minutes
6. Returns `{ "code": "XFER-...", "expires_at_unix": N }`

GET /api/me/cards

Source: `src/cards.rs` -> `my_cards()`

Flow:

1. Requires `CurrentUser`
2. Calls `ensure_card_access()` for metrics only (errors ignored)
3. `SELECT card_number, status, duration_days, activated_at, expires_at, held_reason, created_at FROM access_cards WHERE owner = $1 ORDER BY created_at DESC LIMIT 100`
4. Returns `{ "cards": [...] }`

GET /api/me/cards/summary

Source: `src/cards.rs` -> `my_cards_summary()`

Flow:

1. Requires `CurrentUser`
2. Fetches active card info: `SELECT COUNT(*), MAX(expires_at) FROM access_cards WHERE owner = $1 AND status = 'active' AND expires_at > now()`
3. Fetches banked (unused) card info: `SELECT COUNT(*), COALESCE(SUM(duration_days), 0) FROM access_cards WHERE owner = $1 AND status = 'unused'`
4. Computes: `days_remaining` (from active card), `banked_days`, `total_amount_days`
5. Returns summary object

POST /api/merchant/register

Source: `src/cards.rs` -> `merchant_register()`

Flow:

1. Requires `CurrentUser`
2. `INSERT INTO merchant_profile (user_id) VALUES ($1) ON CONFLICT DO NOTHING`
3. Audit-logs: `merchant.registered`
4. Returns `{ "registered": true }`

POST /api/merchant/cards/purchase

Source: `src/cards.rs` -> `purchase_request()`

Flow:

1. Requires `CurrentUser`
2. Validates merchant status (not banned, not locked out)
3. Validates quantity (1-500)
4. Fetches pricing: `SELECT unit_price_cents, crypto_address FROM billing_config WHERE id = 1`
5. Computes total: `unit_price_cents * quantity`
6. Inserts purchase request: `INSERT INTO card_purchases (id, merchant, quantity, unit_price_cents, total_cents, tx_reference) VALUES (...)`

7. Audit-logs: `merchant.purchase_requested`
8. Returns `{ "purchase_id": "...", "total_cents": N, "crypto_address": "...", "status": "pending" }`

POST /api/merchant/cards/give

Source: `src/cards.rs` -> `give_cards()`

Flow:

1. Requires `CurrentUser` (must be active merchant)
2. Looks up target user by username (case-insensitive)
3. Calls `move_cards()`:
- `WITH pick AS (SELECT id FROM access_cards WHERE creator_merchant = $2 AND owner IS NULL AND status = 'unused' ORDER BY created_at LIMIT $3) UPDATE access_cards c SET owner = $1 WHERE id IN (SELECT id FROM pick)`
- Returns rows_affected count
4. If moved > 0: updates `merchant_profile.total_sold`
5. Audit-logs: `merchant.cards_given`
6. Returns `{ "given": N, "to_user": "..." }`

POST /api/merchant/cards/redeem-transfer

Source: `src/cards.rs` -> `redeem_transfer()`

Flow:

1. Requires `CurrentUser` (must be active merchant)
2. Hashes the provided code, looks up in `card_transfer_codes` (must be unused, not expired)
3. Calls `move_cards()` to deliver cards to the code requester
4. Marks code as used: `UPDATE card_transfer_codes SET used_at = now(), used_by = $2, cards_moved = $3 WHERE id = $1`
5. Audit-logs: `merchant.transfer_redeemed`
6. Returns `{ "delivered": N, "to_user": "..." }`

---

14. Admin Cards (`/api/admin/cards/*`)

POST /api/admin/cards/mint

Source: `src/cards.rs` -> `mint_cards()`

Flow:

1. Requires `StaffUser` with `AdminCards` permission
2. Calls `mint_internal()` to generate cards:
- For each card: generates `SPC-{body}-{hmac8}` number
- Stores: `INSERT INTO access_cards (id, card_number, number_hmac, issued_by, creator_merchant, owner, duration_days) VALUES (...)`
- `number_hmac` is the full HMAC-SHA256 (8-char prefix used in card number, full stored for verification)

3. Returns array of card numbers
4. Audit-logs: `admin.cards_minted`

POST /api/admin/cards/grant

Source: `src/cards.rs` -> `grant_cards_to_user()`

Flow:

1. Requires `StaffUser` with `AdminCards` permission
2. Resolves target user by `user_id` or `username`
3. Validates count (1-100) and duration (1-365 days)
4. Calls `mint_internal()` with `issued_by='admin_grant'` and `owner=target_user_id`
5. Sends notification to the target user via hub
6. Audit-logs: `admin.cards_granted`
7. Increments `cards_admin_granted_total` metric
8. Returns 201 with card numbers

POST /api/admin/cards/{card_number}/hold

Source: `src/cards.rs` -> `hold_card()`

Flow:

1. Requires `StaffUser` with `AdminCards` permission
2. If `hold=true`: `UPDATE access_cards SET status='held', held_reason=$3 WHERE card_number=$1 AND status <> 'held'`
3. If `hold=false`: `UPDATE access_cards SET status='unused', held_reason='' WHERE card_number=$1 AND status='held'`
4. Audit-logs: `admin.card_held` or `admin.card_released`
5. Held cards are immediately dead for playback (the `ensure_card_access` check excludes `status='held'`)

GET /api/admin/cards/chain

Source: `src/cards.rs` -> `card_chain()`

Flow:

1. Requires `StaffUser` with `AdminCards` or `AdminView` permission
2. Joins across `access_cards`, `users` (merchant), `users` (holder)
3. Groups by merchant, holder, and status
4. Returns merchant-to-holder distribution

GET /api/admin/cards

Source: `src/cards.rs` -> `admin_cards_search()`

Flow:

1. Requires `StaffUser` with `AdminCards` or `AdminView` permission
2. Builds dynamic query with optional `status` and `owner` (ILIKE) filters
3. Joins with `users` for owner and merchant usernames
4. Returns paginated card list with total count

GET /api/admin/billing/config

Source: `src/cards.rs` -> `get_billing_config()`

Flow:

1. Requires `StaffUser` with `AdminBilling` or `AdminView` permission
2. `SELECT unit_price_cents, currency, crypto_address FROM billing_config WHERE id = 1`
3. Returns billing configuration

PATCH /api/admin/billing/config

Source: `src/cards.rs` -> `patch_billing_config()`

Flow:

1. Requires `StaffUser` with `AdminBilling` permission
2. Validates price range (10-100000 cents)
3. Builds dynamic UPDATE with only provided fields
4. Executes update
5. Audit-logs: `billing.config_updated`
6. Returns updated config

POST /api/admin/card-purchases/{id}/approve

Source: `src/cards.rs` -> `approve_purchase()`

Flow:

1. Requires `StaffUser` with `AdminBilling` permission
2. Looks up purchase, validates it's `'"pending"'` status
3. Validates merchant is not banned
4. Mints the requested quantity of cards (or `mint_count` if specified)
5. Updates purchase status to `'"approved"'`
6. Updates `merchant_profile.total_purchased`
7. Audit-logs: `admin.purchase_approved`
8. Returns `{ "approved": true, "minted": N }`

POST /api/admin/card-purchases/{id}/reject

Source: `src/cards.rs` -> `reject_purchase()`

Flow:

1. Requires `StaffUser` with `AdminBilling` permission
2. Updates purchase status to `"rejected"` (only if currently `"pending"`)
3. Audit-logs: `admin.purchase_rejected`
4. Returns `{ "rejected": true }`

POST /api/admin/merchants/{user_id}/ban

Source: `src/cards.rs` -> `ban_merchant()`

Flow:

1. Requires `StaffUser` with `AdminMerchants` permission
2. If `banned=true`: `UPDATE merchant_profile SET banned = TRUE, banned_at = now() WHERE user_id = $1`
3. If `banned=false`: clears ban
4. Banned merchants cannot perform any card operations (purchase, give, redeem)
5. Audit-logs: `admin.merchant_banned` or `admin.merchant_unbanned`
6. Returns `{ "banned": true/false }`

POST /api/admin/merchants/{user_id}/lock

Source: `src/cards.rs` -> `lock_merchant()`

Flow:

1. Requires `StaffUser` with `AdminMerchants` permission
2. Sets `locked_out` flag on `merchant_profile`
3. Locked merchants can still read inventory but cannot dispense cards
4. Existing cards held by users remain fully usable
5. Audit-logs: `admin.merchant_locked` or `admin.merchant_unlocked`
6. Returns `{ "locked": true/false }`

---

15. Graph / Media Relationships (`/api/graph/*`)

GET /api/graph/{item_id}/relationships

Source: `src/graph/routes.rs` -> `get_relationships()`

Flow:

1. Requires `CurrentUser`
2. Queries the `media_relationships` table for all relationships involving this item
3. Supports 12 relationship types: sequel, prequel, spin_off, remake, adaptation, related, same_franchise, same_director, same_cast, influenced_by, similar_to, part_of_collection
4. Returns relationships grouped by type with target item metadata

---

GET /api/graph/{item_id}/traverse

Source: `src/graph/routes.rs` -> `traverse_graph()`

Flow:

1. Requires `CurrentUser`
2. Query params: `depth` (1-5, default 2), `relationship_types` (optional filter)
3. Runs BFS traversal starting from the given item
4. At each level: fetches relationships, follows edges to connected items
5. Respects depth limit to prevent exponential blowup
6. Deduplicates visited items
7. Returns a graph structure: `{ "root": {...}, "nodes": [...], "edges": [...] }`

POST /api/graph/{item_id}/relationships

Source: `src/graph/routes.rs` -> `add_relationship()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `target_item_id`, `relationship_type`, optional `metadata`
3. Validates both items exist
4. Prevents self-relationships
5. Inserts (or updates) the relationship: `INSERT INTO media_relationships (id, source_id, target_id, type, metadata) VALUES (...) ON CONFLICT (source_id, target_id, type) DO UPDATE SET metadata = $4`
6. Returns created relationship

DELETE /api/graph/{item_id}/relationships/{rel_id}

Source: `src/graph/routes.rs` -> `remove_relationship()`

Flow:

1. Requires `CurrentUser`
2. Deletes the relationship record
3. Returns `{ "deleted": true }`

---

16. Intelligence (`/api/intelligence/*`)

GET /api/intelligence/insights

Source: `src/intelligence/handlers.rs` -> `get_insights()`

Flow:

1. Requires `CurrentUser`
2. Runs analysis on the user's viewing patterns:

- Peak viewing hours

- Average session duration
- Genre preferences over time
- Completion rates by genre

3. Returns computed insights object

GET /api/intelligence/taste-profile

Source: `src/intelligence/handlers.rs` -> `get_taste_profile()`

Flow:

1. Requires `CurrentUser`
2. Analyzes ratings, completed items, favorites, and watch time
3. Computes taste vectors per genre, decade, mood
4. Returns `{ "genres": { "sci-fi": 0.85, "drama": 0.72, ... }, "decades": {...}, "moods": {...} }`

---

17. Heuristic Engine (`/api/heuristic/*`)

GET /api/heuristic/score/{item_id}

Source: `src/heuristic/handlers.rs` -> `score_item()`

Flow:

1. Requires `CurrentUser`
2. Computes a personalized score for the media item based on:

- User's taste profile alignment
- Similar users' ratings
- Item popularity
- Recency factor
- Genre match strength

3. Returns `{ "score": 0.87, "factors": { "taste_match": 0.9, "popularity": 0.7, ... } }`

GET /api/heuristic/scores

Source: `src/heuristic/handlers.rs` -> `score_multiple()`

Flow:

1. Requires `CurrentUser`
2. Query params: `ids` (comma-separated item IDs)
3. Batch-scores multiple items using the same algorithm as `score_item`
4. Returns `{ "scores": { "id1": 0.87, "id2": 0.65, ... } }`

---

18. Discovery (`/api/discovery/*`)

GET /api/discovery/trending

Source: `src/discovery/handlers.rs` -> `get_trending()`

Flow:

1. Requires `CurrentUser`
2. Calculates trending based on recent activity across all users
3. Weights: views in last 24h (0.5), new ratings (0.2), additions to lists (0.15), reviews (0.15)
4. Returns top 20 trending items

GET /api/discovery/new-releases

Source: `src/discovery/handlers.rs` -> `get_new_releases()`

Flow:

1. Requires `CurrentUser`
2. Queries items added to any library in the last 30 days
3. Sorts by added date
4. Returns paginated list

---

19. Webhooks & Automation (`/api/webhooks/*`, `/api/automation/*`)

GET /api/webhooks

Source: `src/webhooks.rs` -> `list_webhooks()`

Flow:

1. Requires `CurrentUser`
2. `SELECT * FROM webhooks WHERE user_id = $1 ORDER BY created_at DESC`
3. Returns user's configured webhooks

POST /api/webhooks

Source: `src/webhooks.rs` -> `create_webhook()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `url` (callback URL), `events` (array of event types), optional `secret` (for HMAC signing)
3. If no secret provided: generates a random one
4. Inserts webhook: `INSERT INTO webhooks (id, user_id, url, events, secret, active) VALUES (...)`
5. Returns webhook with secret (shown once)

POST /api/webhooks/{id}/test

Source: `src/webhooks.rs` -> `test_webhook()`

Flow:

1. Requires `CurrentUser`
2. Sends a test payload to the webhook URL with HMAC-SHA256 signature
3. Records the response status
4. Returns `{ "success": true/false, "status_code": N }`

DELETE /api/webhooks/{id}

Source: `src/webhooks.rs` -> `delete_webhook()`

Flow:

1. Requires `CurrentUser`
2. Verifies ownership
3. Deletes webhook
4. Returns `{ "deleted": true }`

GET /api/automation/rules

Source: `src/automation.rs` -> `list_rules()`

Flow:

1. Requires `CurrentUser`
2. `SELECT * FROM automation_rules WHERE user_id = $1 ORDER BY created_at DESC`
3. Returns user's automation rules

POST /api/automation/rules

Source: `src/automation.rs` -> `create_rule()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `name`, `trigger` (WHEN condition), `conditions` (IF filters), `actions` (THEN effects)
3. Trigger types: `media_added`, `status_changed`, `scheduled` (cron-like)
4. Condition types: `media_type_is`, `genre_contains`, `rating_above`, etc.
5. Action types: `send_webhook`, `update_status`, `add_to_list`, `send_notification`
6. Inserts rule into `automation_rules`
7. Returns created rule

POST /api/automation/rules/{id}/test

Source: `src/automation.rs` -> `test_rule()`

Flow:

1. Requires `CurrentUser`
2. Runs the rule's conditions against a test media item (or current state)
3. Returns what actions would fire without actually executing them
4. Returns `{ "would_fire": true, "matched_actions": [...] }`

---

20. Extensions & Plugins (`/api/extensions/*`)

GET /api/extensions

Source: `src/extensions/handlers.rs` -> `list_extensions()`

Flow:

1. Requires `CurrentUser`
2. `SELECT * FROM installed_extensions WHERE user_id = $1 ORDER BY name`
3. Returns installed extensions with status

POST /api/extensions/install

Source: `src/extensions/handlers.rs` -> `install_extension()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `manifest_url` or `repo_url`
3. Fetches and validates the extension manifest (Stremio-compatible format)
4. Validates permissions requested
5. Downloads extension code
6. Stores in `installed_extensions` table and filesystem
7. Returns installed extension info

DELETE /api/extensions/{id}

Source: `src/extensions/handlers.rs` -> `uninstall_extension()`

Flow:

1. Requires `CurrentUser`
2. Removes extension files from disk
3. Deletes record from `installed_extensions`
4. Returns `{ "uninstalled": true }`

---

21. Web Sources (`/api/sources/*`)

GET /api/sources

Source: `src/web_sources/handlers.rs` -> `list_sources()`

Flow:

1. Requires `CurrentUser`
2. Lists configured web scraping sources for content discovery
3. Returns source definitions with name, URL pattern, supported types

---

POST /api/sources/search

Source: `src/web_sources/handlers.rs` -> `search_source()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `source_id`, `query`
3. Fetches the source's search URL pattern
4. Makes HTTP request to the external source
5. Parses the response (HTML/JSON depending on source type)
6. Returns parsed results

---

22. HLS Streaming Protection

GET /api/streaming/hls-token

Source: `src/streaming/handlers.rs` -> `generate_hls_token()`

Flow:

1. Requires `CurrentUser`
2. **Card gate:** `ensure_card_access()`
3. Generates a time-limited HLS segment access token
4. Token includes: user_id, allowed qualities, expiry (1 hour)
5. Each HLS segment request must include this token as a query parameter
6. Returns `{ "token": "...", "expires_at": "..." }`

---

23. Friends (`/api/friends/*`)

GET /api/friends

Source: `src/friends.rs` -> `list_friends()`

Flow:

1. Requires `CurrentUser`
2. `SELECT u.id, u.username, u.display_name, u.avatar_url FROM friends f JOIN users u ON u.id = f.friend_id WHERE f.user_id = $1 AND f.status = 'accepted'`
3. Returns friend list

POST /api/friends/request

Source: `src/friends.rs` -> `send_request()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `user_id` (target)
3. Prevents self-request
4. Checks no pending request exists in either direction
5. Inserts: `INSERT INTO friends (id, user_id, friend_id, status) VALUES ($1, $2, $3, 'pending')`
6. Creates notification for target user
7. Returns `{ "sent": true }`

POST /api/friends/accept/{id}

Source: `src/friends.rs` -> `accept_request()`

Flow:

1. Requires `CurrentUser`
2. Verifies the request is directed at the current user and is in `'pending'` status
3. Updates status to `'accepted'`
4. Creates reciprocal friend record if not exists
5. Notifies the requester
6. Returns `{ "accepted": true }`

DELETE /api/friends/{id}

Source: `src/friends.rs` -> `remove_friend()`

Flow:

1. Requires `CurrentUser`
2. Deletes the friend record (both directions)
3. Returns `{ "removed": true }`

---

24. Sharing (`/api/sharing/*`)

POST /api/sharing/link

Source: `src/sharing.rs` -> `create_share_link()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `media_item_id`, optional `expires_in_hours`
3. Generates a unique share code
4. Inserts: `INSERT INTO share_links (id, user_id, media_item_id, code, expires_at) VALUES (...)`

5. Returns `{ "url": "https://.../s/{code}", "expires_at": "..." }`

GET /api/sharing/link/{code}

Source: `src/sharing.rs` -> `resolve_share_link()`

Flow:

1. No authentication required (public endpoint)
2. Looks up share link by code, validates not expired
3. Increments view count
4. Returns the media item details (or redirect URL)

---

25. Moderation (`/api/moderation/*`)

GET /api/moderation/reports

Source: `src/moderation.rs` -> `list_reports()`

Flow:

1. Requires `StaffUser` with `AdminModeration` permission
2. `SELECT * FROM moderation_reports WHERE status = 'pending' ORDER BY created_at DESC`
3. Returns pending reports

POST /api/moderation/report

Source: `src/moderation.rs` -> `submit_report()`

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `target_type` (review/comment/user), `target_id`, `reason`, optional `description`
3. Prevents duplicate reports from same user on same target
4. Inserts report into `moderation_reports`
5. Returns `{ "reported": true }`

POST /api/moderation/reports/{id}/resolve

Source: `src/moderation.rs` -> `resolve_report()`

Flow:

1. Requires `StaffUser` with `AdminModeration` permission
2. Deserializes JSON: `action` (dismiss/warn/suspend/ban), optional `note`
3. Updates report status to `"resolved"`
4. If action is warn/suspend/ban: applies the action to the target user

5. Audit-logs the resolution
6. Returns `{ "resolved": true }`

---

26. Backup & Restore

POST /api/admin/backup

Source: Admin routes

Flow:

1. Requires `StaffUser` with `AdminBackup` permission
2. Triggers a database backup via `pg_dump` or equivalent
3. Stores backup file in configured backup location
4. Returns `{ "backup_id": "...", "size_bytes": N }`

POST /api/admin/restore

Source: Admin routes

Flow:

1. Requires `StaffUser` with `AdminRestore` permission
2. Accepts a backup file upload
3. Validates backup integrity
4. Restores database from backup (dangerous operation, requires confirmation)
5. Returns `{ "restored": true }`

---

27. Realtime / WebSocket (`/ws/*`)

WebSocket /ws

Source: `src/realtime/` or hub module

Flow:

1. Upgrades HTTP connection to WebSocket
2. Authenticates via token in query parameter or first message
3. Registers connection in the hub's connection registry
4. Enters event loop:
- **Incoming:** processes messages (chat, sync, presence, typing indicators)
- **Outgoing:** delivers broadcast events (new messages, notifications, presence changes, sync commands)
5. Handles ping/pong for keepalive (30-second interval)
6. On disconnect: removes from registry, updates presence to offline

---

28. Telemetry & Metrics (`/metrics/*`)

GET /metrics

Source: Internal metrics endpoint

Flow:

1. Requires admin authentication
2. Collects metrics from the `metrics` crate (Prometheus format):
- `playback_started_total` -- number of playback sessions started
- `stream_started_total` -- number of streams served
- `cards_activated_total` -- card activations
- `card_activation_failures_total` -- failed activation attempts
- `card_activation_lockouts_total` -- lockout events
- `playback_blocked_no_card_total` -- blocked playback attempts
- `http_requests_total` -- all HTTP requests by route and status
- `http_request_duration_seconds` -- request latency histogram
3. Returns Prometheus text format

GET /health

Source: Health check endpoint

Flow:

1. No authentication required
2. Checks database connectivity: `SELECT 1`
3. Checks hub status (connected clients count)
4. Returns `{ "status": "healthy", "database": "ok", "uptime_seconds": N, "connected_clients": N }`

---

29. Cache Warming

POST /api/admin/warm-cache

Source: `src/warming.rs`

Flow:

1. Requires `StaffUser` with `AdminSystem` permission
2. Triggers background cache warming:
- Pre-computes recommendations for active users
- Pre-fetches trending/new-release lists
- Pre-generates HLS manifests for popular content
3. Runs as a background tokio task
4. Returns `{ "started": true }`

---

30. Security (`/api/security/*`)

GET /api/security/grants

Source: `src/security/routes.rs`

Flow:

1. Requires admin
2. Lists active security grants and signing keys
3. Returns grant metadata (not secrets)

POST /api/security/rotate-keys

Source: `src/security/routes.rs`

Flow:

1. Requires admin
2. Generates new signing keys for the specified grant type (cards, webhooks, etc.)
3. Old keys remain valid for a grace period (7 days)
4. Returns `{ "rotated": true, "new_key_id": "..." }`

GET /api/security/risk-assessment

Source: `src/security/risk.rs`

Flow:

1. Requires admin
2. Analyzes risk factors across the system:
- Failed login attempts
- Card activation lockouts
- Unusual streaming patterns
- API rate limit hits
3. Returns risk score and flagged events

---

31. Librarian View (`/api/librarian/*`)

GET /api/librarian/library-stats

Source: Catalog routes

Flow:

1. Requires `CurrentUser`
2. Computes library statistics:
- Total items by type
- Total file size
- Average rating
- Items by decade
- Genre distribution
3. Returns comprehensive stats object

GET /api/librarian/missing-metadata

Source: Catalog routes

Flow:

1. Requires `CurrentUser`
2. Finds items with incomplete metadata (missing poster, description, genres)
3. Returns list of items that could benefit from metadata import

---

32. Smart Collections

GET /api/tracking/smart-collections

Source: Tracking routes

Flow:

1. Requires `CurrentUser`
2. Generates dynamic collections based on user's data:
- "Unwatched" -- items with status=planned or no progress
- "Rewatchable" -- items rated >= 8
- "Recently Finished" -- completed in last 30 days
- "Genre Collections" -- grouped by top genres
3. Each collection is computed from existing tracking data (no database writes)
4. Returns collections with item counts and previews

---

33. Labels (`/api/labels/*`)

GET /api/labels

Source: Label routes

Flow:

1. Requires `CurrentUser`
2. Returns user's custom labels (colored tags for organizing media)
3. `SELECT * FROM user_labels WHERE user_id = $1 ORDER BY name`

POST /api/labels

Source: Label routes

Flow:

1. Requires `CurrentUser`
2. Deserializes JSON: `name`, `color` (hex string)
3. Inserts into `user_labels`
4. Returns created label

POST /api/labels/{label_id}/items/{item_id}

Source: Label routes

Flow:

1. Requires `CurrentUser`
2. Associates a label with a media item
3. `INSERT INTO label_items (label_id, media_item_id) VALUES ($1, $2) ON CONFLICT DO NOTHING`
4. Returns `{ "labeled": true }`

---

34. Lists (`/api/lists/*`)

GET /api/lists

Source: List routes

Flow:

1. Requires `CurrentUser`
2. Returns all user lists with item counts
3. Includes system lists (Favorites, Watchlist, Custom)

POST /api/lists

Source: List routes

Flow:

1. Requires `CurrentUser`
2. Creates a new named list
3. Returns created list

GET /api/lists/{id}

Source: List routes

Flow:

1. Requires `CurrentUser`
2. Returns list details with all items (joined with media_items for metadata)
3. Items ordered by `position` in the list

---

35. Hub (`/api/hub/*`)

GET /api/hub/status

Source: Hub module

Flow:

1. Requires `CurrentUser`
2. Returns hub connection status, connected client count, and message queue stats
3. Used for monitoring the realtime system

---

36. Warming (`/api/admin/warming/*`)

GET /api/admin/warming/status

Source: `src/warming.rs`

Flow:

1. Requires admin
2. Returns status of background warming tasks:
- Recommendations cache: last warmed, next scheduled
- Trending cache: status
- HLS manifest cache: status
3. Returns `{ "recommendations": { "last_warmed": "...", "users_cached": N }, ... }`

Appendix: Common Patterns

Card Gating

Most streaming and playback endpoints call `ensure_card_access(pool, user_id)` which runs:

```
SELECT EXISTS (
  SELECT 1 FROM access_cards
  WHERE owner = $1 AND status = 'active' AND expires_at > now()
)
```

If false, returns 403 "no active access card". This is the primary monetization gate.

Authentication Middleware

Two layers:

1. `CurrentUser` -- extracted from JWT Bearer token. Contains `user_id`, validated against `revoked_tokens`.
2. `StaffUser` -- extends `CurrentUser` with role-based permission checks. Calls `staff.require("permission_name")`.

Audit Logging

Most state-changing operations call `crate::audit::log()` or `crate::audit::log_simple()` which inserts into an `audit_log` table with: user_id, action, target, metadata, ip_address.

Metrics

The `metrics` crate is used throughout. Common counters:

- `http_requests_total` (auto by axum middleware)
- `playback_started_total`, `stream_started_total`
- `cards_activated_total`, `card_activation_failures_total`

- `playback_blocked_no_card_total`

Error Handling

All handlers return `ApiResult>` which maps `ApiError` variants to HTTP status codes:

- `ApiError::NotFound` -> 404
- `ApiError::Forbidden` -> 403
- `ApiError::TooManyRequests` -> 429 (with retry-after)
- `ApiError::BadRequest` -> 400
- `ApiError::Unauthorized` -> 401
- `ApiError::Internal` -> 500